

EPIISODE 6

Microservices, Scaling & Web3 Basics

Distributed Systems, ECS, Blockchain

Monolith vs Microservices

API Gateway Pattern

RabbitMQ Events

AWS ECS & Fargate

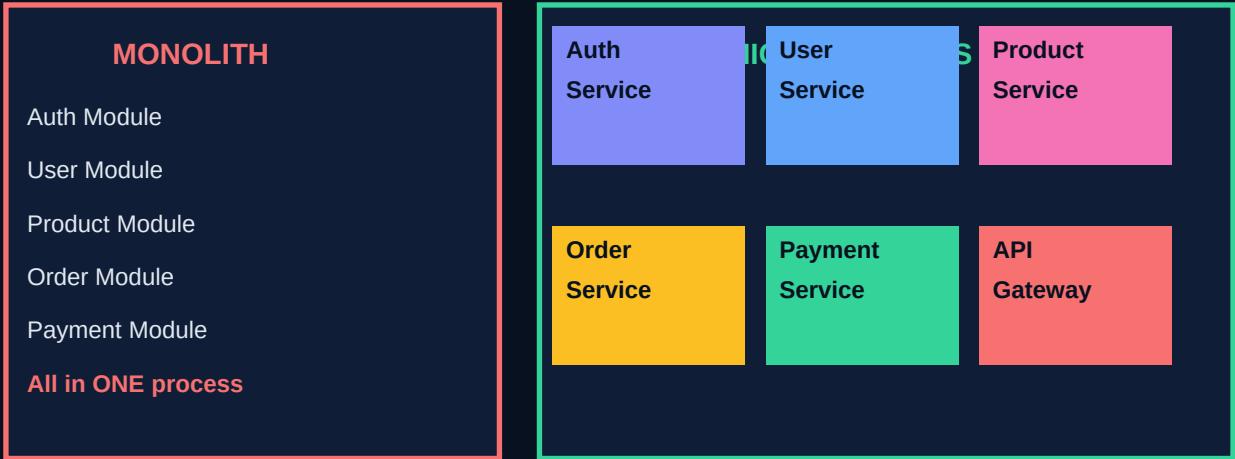
Docker Multi-service

Web3 & Blockchain

CHAPTER 1 — MONOLITH vs MICROSERVICES

Microservices Architecture

A monolith bundles everything into one deployable unit. Microservices split by domain — each service is independently deployable, scalable, and owned by a team. Use microservices when teams are large, services need different scaling, or different tech stacks are needed. Don't use them for small apps — the operational overhead is real.

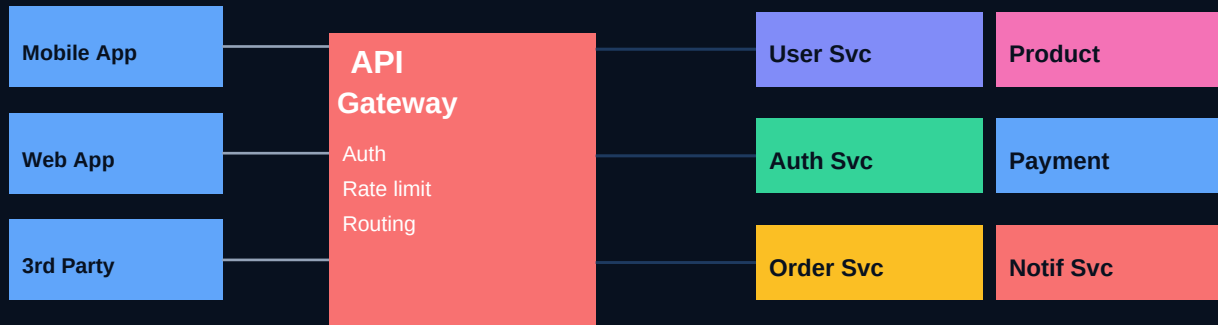


Monolith: simple to start. Microservices: scalable but complex — choose carefully

Aspect	Monolith	Microservices
Deploy	One unit	Each service independently
Scaling	Scale whole app	Scale bottleneck service only
Teams	One codebase	Each team owns a service
Tech stack	One language	Polyglot (different per service)
DB	Shared database	DB per service
Failure	One bug can crash all	Isolated failures
Start complexity	Low	High — networks, auth, tracing

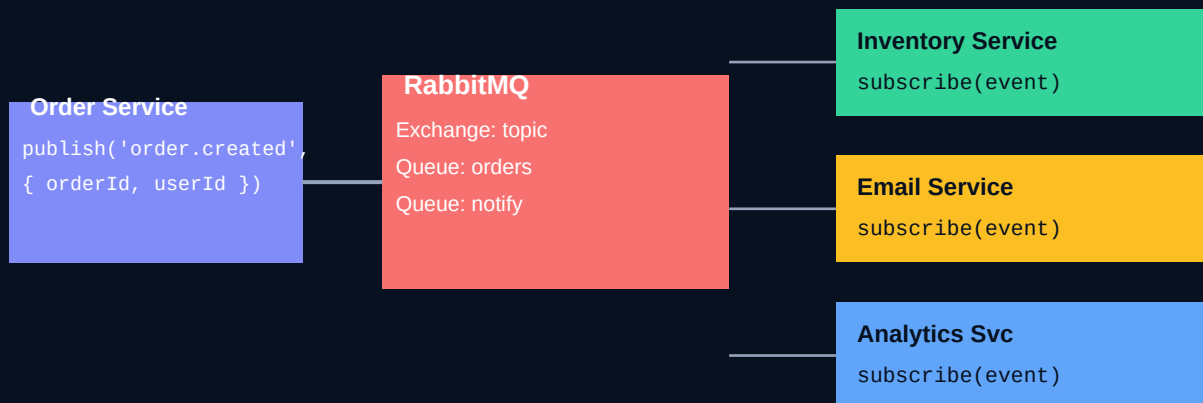
CHAPTER 2 — API GATEWAY & EVENT-DRIVEN

API Gateway Pattern



API Gateway: single entry point for all clients — handles auth, rate limiting, routing

RabbitMQ Event-Driven Architecture



Message queue: services communicate asynchronously — loose coupling, high resilience

```

// RabbitMQ publisher (Order Service)
const amqp = require('amqplib');
const conn = await amqp.connect(process.env.RABBITMQ_URL);
const ch = await conn.createChannel();
await ch.assertExchange('orders', 'topic', { durable: true });
ch.publish('orders', 'order.created', Buffer.from(JSON.stringify({
  orderId: order._id, userId: order.userId, total: order.total
})));

// RabbitMQ consumer (Email Service)
const ch2 = await conn.createChannel();
await ch2.assertQueue('email_notifications', { durable: true });
await ch2.bindQueue('email_notifications', 'orders', 'order.created');
ch2.consume('email_notifications', async (msg) => {
  const event = JSON.parse(msg.content);
  await sendOrderConfirmationEmail(event.userId, event.orderId);
  ch2.ack(msg); // acknowledge - remove from queue
});
  
```

CHAPTER 3 — AWS ECS & FARGATE

Container Orchestration on AWS

Service	Purpose	Use When
ECR	Store Docker images	Always — private container registry
ECS	Run containers on AWS	When you need container orchestration
Fargate	Serverless containers	No EC2 management needed
EC2 Launch Type	Containers on your EC2	Need cost control, GPU, or specific config
ALB	Load balance ECS tasks	Multiple containers need traffic distribution
CloudWatch	Monitor ECS tasks	Always — logs & metrics for containers

CHAPTER 4 — WEB3 BASICS

Blockchain & Decentralized Web

Concept	What It Is	Example
Blockchain	Immutable distributed ledger	Ethereum, Bitcoin, Solana
Smart Contract	Self-executing code on blockchain	Uniswap, OpenSea contracts
DApp	App with blockchain backend	Uniswap, Aave, MetaMask
Token/NFT	Blockchain-based digital asset	ERC-20 tokens, ERC-721 NFTs
Wallet	Private key container	MetaMask, Phantom
Web3.js	JS library for blockchain	ethers.js, web3.js

```
// Connecting to Ethereum with ethers.js
import { ethers } from 'ethers';

// Connect to MetaMask wallet
const provider = new ethers.BrowserProvider(window.ethereum);
await provider.send('eth_requestAccounts', []);
const signer = await provider.getSigner();
const address = await signer.getAddress();

// Read from a smart contract
const contract = new ethers.Contract(CONTRACT_ADDRESS, ABI, provider);
const balance = await contract.balanceOf(address);

// Write to a smart contract (costs gas)
const tx = await contract.connect(signer).transfer(toAddress, amount);
await tx.wait(); // wait for confirmation
```

Episode 6 Summary

- ✓ Monolith vs Microservices — trade-offs, when to use each
- ✓ API Gateway — centralized auth, rate limiting, routing
- ✓ RabbitMQ — async event-driven communication between services
- ✓ AWS ECS + Fargate — managed container orchestration
- ✓ Web3 Basics — blockchain, smart contracts, DApps, ethers.js